

Chio: A Peer-to-Peer Economy of Verifiable Work

The Chio Project

<https://github.com/bb-connor/arc>

October 2026

Abstract

An agentic web needs programs that recruit collaborators and grow across administrative boundaries while each owner retains control over its resources. We present Chio, a kernel interface for composing work through local authorization and durable commitments. Its organizing rule is that plans may evolve while issued commitments retain their authority bounds and execution identity. Delegated work contracts subdivide an owner's allocation, select signed offers and seal the exact receiver, input and invocation into portable evidence. For running swarms, signed graphs grow by retaining earlier tasks, allocations and continuation identities. Each receiver applies its own capabilities and treaty conditions before dispatch. Under protected allocation and receiver custody, the composition conserves declared capacity across graph versions and prevents graph growth from refreshing a consumed invocation. A Rust implementation demonstrates recursive delegation, collaborator replacement before commitment, live swarm extension, checked output and recovery after a process kill. Native tests preserve unfinished work and original receipts across extension and restart. A separately evaluated funded profile preserves recorded child payment claims through parent failure. These results establish a concrete execution boundary for evolving work relationships: local authority governs admission, and durable commitments govern what survives a change of plan.

1 Programs across independent owners

The agentic web calls for programs whose participants need not share an owner, a scheduler or a lifetime. An agent may assemble a research service, a software builder and a verifier only after discovering what a task requires. Each participant must be able to cooperate while retaining control over its own tools, data and obligations. The programming problem is therefore both dynamic composition and preservation of authority through change.

An owner authorizes an agent to complete a task within a budget and a set of permitted actions. The agent then discovers that it needs a specialist. That specialist may need another collaborator. Which of these choices can be made without asking the owner to authorize an entirely new system? What prevents the same allowance from being promised to two providers? If a provider stops responding, which work can continue and which obligation must remain intact?

These questions belong at the boundary between application planning and resource control. The planner decides what work to request and from whom. The kernel must preserve the owner's constraints when those decisions change. A fixed workflow can encode each relationship separately. An agentic operating system needs a reusable operation for forming and extending such relationships at runtime. Across organizations, that operation must also respect each receiver's own authority: a remote contract cannot grant itself access to local resources.

Consider an owner allocating 100 units to a work tree. A holder assigns up to 60 to a specialist branch and 40 to an independent sibling. The first branch can select among qualified providers before commitment. Once its selection is sealed, losing the response cannot justify spending the same allocation with a different provider. The sibling can still proceed from its separate share. Recovery follows the first branch’s original identity. This is the implemented delegation problem; it does not require an LLM to be trusted to track the budget.

Chio expresses this boundary with a *work slot*, authenticated subdivision, a signed provider offer and a sealed dispatch permit. The permit is portable: the receiver can check it without querying the owner’s allocator. Exclusive allocation still requires protected state at that allocator. A receiver admits the exact invocation under its own capability, retains its execution identity, and checks the exact output before release. The resulting interface separates the freedom to choose collaborators from the authority to exceed an allocation.

The economic layer adds a different requirement: a promised payment must be backed by funds or another explicitly qualified source of credit. A signed budget is insufficient. In our separately evaluated funded profile, a buyer reserves 100 and an intermediary separately reserves 60 for a specialist. The specialist’s recorded earned claim survives a failed parent. The two profiles distinguish authority allocation from financial backing; an allocation certificate is never silently treated as escrow.

The same distinction applies when a running swarm discovers additional work. A replacement graph must preserve earlier promises even if its planner has changed its mind or has lost a response. Chio extends its existing signed swarm graphs by retaining every issued allocation and continuation identity. An atomic head transition admits new tasks within the original envelope. Historical graph versions remain available for old requests, and the native kernel retains custody of every consumed continuation across versions. Bilateral treaties continue to govern local admission.

The contribution is a concrete composition rule: *editable plans with preserved commitments*. We give its conditional conservation argument, implement it using existing authority and recovery machinery, and exercise the boundary at native dispatch. Capability systems and secure linking provide substantial antecedents [1, 2]; ordinary transactional and agent-commerce constructions match the tested monetary outcomes. Bitcoin obtains a shared spend history under its consensus assumptions [3]; Chio relies on declared allocation and settlement authorities. The scientific hypothesis is that this common boundary can make independently operated work systems materially easier to compose. The implementation makes that hypothesis testable through integration cost and useful-work outcomes.

2 Principals, resources and assumptions

The *owner* configures an allocator and creates a root contract. A *holder* may subdivide its allocation and select an offer. A *receiver* offers a tool operation and issues the native capability under which it will run. A courier transports signed records. An organization can occupy several roles; distinct keys alone do not establish independent administration. Holders and receivers in the dynamic profile are previously qualified keys explicitly permitted by the owner. Discovery selects among them; it does not enroll an unknown party or authorize a new disclosure.

Agents and couriers may be malicious. Messages may be delayed, duplicated, reordered or lost. Processes may crash between durable steps. The honest owner’s allocator serializes allocation and preserves its state. Each receiving key has one protected native custody and budget domain. Cloning those keys and databases into independently spendable forks violates the assumptions. Signatures

authenticate statements; they do not prevent a trusted issuer that has been compromised from making contradictory statements.

We distinguish three experimental profiles. **D1** implements dynamic allocation and locally checked dispatch permits. It assumes honest allocator enforcement, collision-resistant hashes, unforgeable signatures, accurate validity checks, and the receiver’s capability, durable admission and payment enforcement. Its effect and reader sets are declarations whose completeness depends on the qualified host and tool confinement. Set inclusion alone does not prove information-flow security or prevent an arbitrary tool from using an undeclared channel. The local implementation uses a durable integer payment fixture; its budget allocations are ceilings, not proof of deposited assets.

S1 implements additive evolution of Chio’s existing swarm authority. It uses signed task graphs, attenuation witnesses, routes, revocation epochs, allocations and single-use continuations. Each graph has one protected current head, and one protected authority controls the declared pool across that graph’s versions. The installer serializes successors; receivers resolve the exact version named by a request and preserve native continuation custody. Reusing a pool in independently provisioned graphs or stores is outside this allocation discipline. The evaluated graph growth is additive: it preserves permissions, expiry, declared capacity and prior artifacts instead of reclaiming or transferring them. Ordinary capability and treaty checks apply at dispatch.

F1 implements funded, fixed-price work claims. A payer B reserves funds for provider P in settlement domain R . A selected verifier V applies the agreed predicate to bound inputs and outputs. F1 additionally assumes that the rail enforces exclusive allocations and atomic transfers, the asset transfers the agreed units, and the finality observation is sound. Collection requires available evidence, eligible verifier decisions and transaction inclusion. The verifier remains a trusted decision authority even if others can rerun its checker. D1, S1 and F1 share architectural boundaries but have separate evaluated trajectories. No reported execution combines all three into one dynamic, funded, independently operated swarm.

Three kinds of state answer different questions. Allocation state records which capacity is committed. Native custody records what was admitted and observed during execution. Settlement state records what is reserved, payable and transferred. An unknown tool effect remains unknown after a financial refund. A sealed allocation remains committed after a network timeout. A historical receipt cannot authorize a new invocation.

The bounds concern operations admitted under these profiles. They exclude off-protocol spending, opportunity cost, loss of disclosed data and fees unless those costs are explicitly reserved. D1 supports finite trees with bounded wire objects; S1 grows finite signed graphs; F1 evaluates distinct funded edges. These profiles do not prove that an arbitrary task decomposes correctly, that every participant remains available or that the work is economically useful.

3 The work contract

Write a D1 slot as

$$S = (i, h, E, A, b, c, t, d, Q), \quad s = H(\text{canon}(S)).$$

i identifies the slot, h its holder, E its exact server/tool pairs, A its permitted reader keys, b its ceiling in currency c , t its expiry, d its remaining depth and Q its acceptance predicate. The owner creates a root through a trusted configuration API. Each allocator has a persistent namespace N . The allocation digest $a = H(\text{canon}(N, S_{\text{root}}, S))$ binds both the full root and the target slot within that namespace, under a versioned domain. A holder signs a subdivision naming the parent, its

allocation digest and the complete child. The allocator admits it only when that digest matches its protected state and

$$\begin{aligned} E_{\text{child}} &\subseteq E_{\text{parent}}, & A_{\text{child}} &\subseteq A_{\text{parent}}, \\ t_{\text{child}} &\leq t_{\text{parent}}, & d_{\text{child}} &< d_{\text{parent}}, \\ c_{\text{child}} &= c_{\text{parent}}, & \sum_{j \in \text{children}(S)} b_j &\leq b_S. \end{aligned}$$

The child must still be live. Its holder and selected receiver must be in its reader set. Immediate SQLite transactions couple insertion with the parent’s allocation counter. A slot with children cannot execute; a selected slot cannot acquire children. Unused internal capacity remains available for additional children. There is no reclaim operation in D1.

Children perform different subtasks and may have different predicates. A successful child therefore does not establish that its parent task succeeded. A separate leaf may assemble results and check the final predicate within its own allocation. D1 supplies capacity-preserving subdivision, not automatic semantic decomposition of an owner’s goal.

3.1 Offer, selection and commitment

A receiver signs an offer binding the allocation digest a , complete slot digest s , its own key, the exact effect and input digest, a price ceiling and an expiry. Binding the complete slot is essential: signing only its name would allow the offer to be transplanted into different acceptance terms. Binding a additionally prevents replay between honest owners using equal local slot names and terms. The holder signs a selection containing that offer, the original native request ID, the receiver-issued capability digest and the expected selection revision. Compare-and-swap revisions permit replacement and reject stale selections.

Sealing freezes the selected binding before the allocator exports a signed *dispatch permit*. The permit contains the root ID, allocation digest, immutable slot, signed selection and issuance time. An accepted allocator signature attests that the slot was allocated and sealed; the receiver does not independently reconstruct the owner’s entire allocation tree. It checks this evidence against its configured allocator keys, while its native capability supplies the actual authority to execute.

Every signed body has its own versioned domain and uses RFC 8785 canonical JSON [4]. The input digest covers the slot ID and exact payload. It excludes the enclosing permit to avoid circular hashing; the permit has its own signature. Unknown wrapper fields reject. Wire numbers are limited to $2^{53} - 1$, depth to 32, children per slot to 64 and slots per root to 4,096. The acceptance language is a conjunction of 1–16 exact JSON-pointer equalities or inclusive integer ranges. These limits define the implemented profile, rather than a general language for proving useful computation.

3.2 An allocation ceiling and a funded obligation

F1 binds an agreement digest, payer, beneficiary, verifier, asset, amount and deadlines to an actual reserved allocation. The allocation identifier includes the chain and contract address. A new payer agreement cannot be funded twice under the same payer and digest. The receiver relies on a qualified observation of that reserve, not a signed balance supplied by the payer.

Acceptance in F1 is a separate EIP-712 decision [5] binding the allocation, agreement, submission, decision digest, acceptance bit, beneficiary, asset and amount. The escrow authenticates the pinned

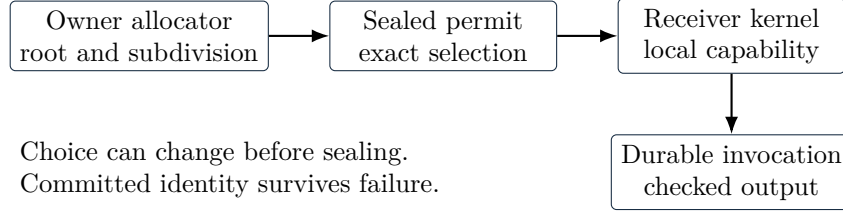


Figure 1: D1 separates coordinated allocation from locally checked execution. After sealing, the receiver needs its own custody and payment services but no live allocator connection. The permit grants no native capability.

verifier; it does not execute the application checker. A missing output or custody record yields no decision. An available result that fails the predicate yields rejection.

A child agreement is separately funded. In the 100/60 example, the intermediary must already control the child’s 60; the buyer’s reserved 100 cannot be pledged again. D1’s 100-unit allocation tree, by comparison, divides one owner’s authority ceiling. It does not promise both a 100-unit parent refund and additional child payments from the same 100. Credit, conditional netting and cross-profile deployment require their own backing and integration rules.

4 Execution, recovery and payment

4.1 The native delegation boundary

D1’s receiver consumes a slot ID, payload and signed dispatch permit. The installed guard requires a locally accepted allocator and checks the allocator, holder and receiver signatures. It compares the exact receiver key, capability subject, server/tool, input digest, request ID and capability digest with the sealed selection. The capability must be issued by this receiving kernel and contain one exact invoke grant, one invocation, and per-call and total monetary ceilings no larger than the offer. The offer and slot must be live. Ordinary native capability validation, revocation and policy checks still apply.

The installation API requires durable native admission and enables original request retention. The guard revalidates before dispatch and checks the exact JSON value after output transformations. A signed offer in D1 agrees to accepted-output pricing: a qualifying predicate rejection withholds output and uses the native reversible-hold path to produce a zero-charge denial. The executed invocation remains consumed. This rule depends on the qualified payment profile; a portable permit does not implement settlement by itself.

Sealing uses two durable steps: first freeze the exact selection, then persist the signed permit before returning it. A crash between them can strand capacity. Retrying returns the original stored permit once available, and cannot replace its provider or request. Native custody, rather than a permit cache, determines whether that request has already executed. Timeouts do not authorize a replacement. Independent siblings can proceed from their own allocations while the original operation is unresolved.

In the native fault experiment, the receiver process is killed after `ToolReturnRecorded`. A second receiver cannot replace the original allocation, but completes an already allocated sibling. Reopening the first receiver recovers its recorded output without a fresh tool call. This cut has recoverable evidence; it does not establish recovery from every possible cut. An effect lacking authoritative

completion evidence remains unknown. Ordinary external effects acquire no exactly-once guarantee from the permit.

4.2 Extending a running swarm

S1 adds one operation to the existing swarm lifecycle. Given the expected digest of the installed bundle and a signed candidate, the runtime store checks both bundles with the ordinary live-authority verifier at store-owned time. It then checks retention: the original graph envelope, nodes, edges, joins, routes, witness chains, allocations and revocation epoch remain intact. The candidate must add nodes within the original depth, fan-out and pool limits. Each task has at most one allocation and continuation. Existing continuations retain every field except the new graph digest and its signature. A new task receives a new continuation; an old task cannot receive a new attempt identity.

In one immediate SQLite transaction, the installer compares the expected head, archives the exact parent bundle and installs the successor. Competing proposals against one parent cannot both commit. The archive binds the graph index to the signed graph digest and full bundle digest. Admission uses the graph digest already carried by the request, so an unfinished operation can still present its original evidence after growth. Every fresh invocation also passes the ordinary current capability, revocation and treaty gates.

The native kernel owns the single-use continuation resource by its stable ID. A changed graph digest therefore cannot turn a consumed continuation into fresh authority. Replaying a completed request resolves its original operation and receipt. Revalidating an original physical claim reads its original graph and treaty evidence. The installer does not need a joint transaction with every running operation: additive growth leaves their allocations and identities intact. Non-additive retirement would require a different authority boundary.

4.3 The separately funded profile

F1's financial state machine has seven states after funding:

$$\text{Funded} \rightarrow \text{Submitted} \rightarrow \begin{cases} \text{Payable} \rightarrow \text{Paid}, \\ \text{Rejected} \rightarrow \text{Refunded}. \end{cases}$$

A funded or submitted job without a decision can become TimedOut and then Refunded. There is no edge from Payable to timeout or refund.

Let $t_s < t_c < t_r < t_f$ denote submission, challenge, resolution and refund boundaries. The beneficiary submits by t_s . A first verifier decision is recorded only when $t_c < t \leq t_r$. Without a decision, refund is allowed only when $t > t_f$. The interval before decision is a timing boundary, not a claim that F1 implements a public challenge court. The exact verifier decision may be carried by any courier. Its first valid recording determines acceptance; an identical replay changes nothing, and a conflicting replay fails.

A claim is *earned* here when an accepting decision has been recorded in the settlement state under the selected finality rule. Merely producing an output or obtaining an off-chain signature does not earn this right. After that transition, the beneficiary may withdraw without a new payer signature. The reference contract imposes no later expiry on an earned withdrawal. Administrative pause and verifier-registry changes govern new funding and do not revise an existing claim. The recipient still needs transaction inclusion, fees and a functioning asset contract.

Local execution proceeds under its own durable operation identity. The kernel commits admission before dispatch, retains observed output and produces signed evidence. The funded integration carries the original allocation, operation, hold and authorization identifiers through verification and recovery. Recovery reopens that history; it must not mint a fresh admission simply because a response was lost. A return whose occurrence cannot be established remains an unknown terminal. A later financial resolution has its own authority and does not rewrite the original observation.

For actual rail transactions, the research implementation records the exact signed transaction before broadcast. After a crash it reobserves or rebroadcasts those same bytes. A changed nonce, transaction or funding context is not an ordinary retry. This approach handles particular durable cutpoints; it is not an exactly-once guarantee for arbitrary tool effects.

4.4 Successful exchange and failed intermediary

In a successful exchange, the buyer and provider sign the terms, the buyer funds f_1 , and the provider admits one bound operation. The result and custody record identify the exact submitted bytes. The verifier reruns the selected check, records acceptance, and the provider withdraws 100. An offline reader can verify the signed evidence without possessing either party's private keys; current spendability still requires settlement state.

For the failed intermediary, the specialist's 60-unit child claim becomes **Payable** but remains uncollected. The parent produces no eligible submission and its 100 returns to the buyer after timeout. The specialist then withdraws the original 60. The provider bears the loss because it funded the child. This is neither a transfer of the buyer's risk to an unpaid specialist nor a claim that failed computation is free.

If instead the verifier never records a child decision, the child may time out and refund its payer despite the specialist having performed work. F1 protects recorded earned claims; it cannot guarantee timely adjudication against its own failed verifier. That distinction is the decisive difference between financial safety and an unconditional promise to pay for every useful result.

5 What composes

The arguments concern transitions under the assumptions of Section 2. They do not prove the complete Rust implementation or semantic correctness of a task decomposition.

5.1 Delegation invariants

Proposition 1 (Declared attenuation and bounded capacity). *In D1, every descendant's declared effects and readers are subsets of its ancestors', its expiry is no later, and the sum of executable leaf ceilings is at most the root ceiling.*

Proof. Subset inclusion and the expiry order are transitive. Strictly decreasing depth bounds the lineage. Let U_v be the unallocated remainder of an internal slot. Initially the root is one leaf of ceiling B . Subdivision moves b units from a leaf or internal remainder into a child of ceiling b , atomically and with a nonnegative remainder. Therefore

$$\sum_{\text{leaves } v} b_v + \sum_{\text{internal } v} U_v = B.$$

Selection and sealing change no ceiling. Containers cannot execute, so only leaf capacity can authorize invocations. Native enforcement of the bound one-invocation capability in one protected receiver custody domain bounds admitted charges by those ceilings. This last step assumes native enforcement; the arithmetic alone does not prove it. $\square$

Proposition 2 (Stable sealed identity). *After a D1 selection is sealed, an accepted permit cannot change its receiver, capability, input, effect or request identity without violating an authenticated binding or the trusted allocator’s transition rules.*

Proof. Selection revisions serialize replacements before sealing. The allocator freezes the selection before exporting a permit and offers no later transition that replaces it. Each receiver compares its invocation with that signed selection and the complete slot digest. Substitution either fails a comparison or requires a new trusted statement. Replaying identical evidence reaches native custody under the original identity; it supplies no fresh capability. $\square$

This result permits changing collaborators before commitment while preserving uncertain obligations afterward. It trades reclaim and automatic failover for a simple safety rule. A future reclaim protocol would need authoritative closure or fencing of the original operation. Expiry or an agent’s claim of absence is insufficient. These D1 arguments are not mechanized; the retained Lean results below concern the separate funded state machine.

5.2 Commitments across graph versions

Let $G_0, \dots, G_n$ be the installed S1 lineage and A_i its set of uniquely identified allocations, with ceiling $b(a)$ for allocation a . Let B be the fixed declared pool. Every checked extension requires

$$A_i \subseteq A_{i+1}, \quad b_{i+1}(a) = b_i(a) \ (a \in A_i), \quad \sum_{a \in A_{i+1}} b(a) \leq B.$$

Proposition 3 (Bounded commitments under additive growth). *Within one protected S1 lineage, the ceilings of all allocations ever installed sum to at most B . Graph extension preserves an old task’s authority artifacts and cannot refresh its consumed native continuation.*

Proof. An atomic compare-and-swap admits at most one successor from each installed head. Retention makes the allocation sets monotone, so $\bigcup_{i=0}^n A_i = A_n$. The live verifier bounds the sum in A_n by B ; checked integer arithmetic prevents overflow. The extension check retains every old continuation ID and its allocation and forbids a second continuation for that task or allocation. Native custody uses this retained ID across graph versions. Re-signing its new graph digest cannot create a second native resource. Exact historical resolution supplies the original artifacts to old requests and physical-claim revalidation. Ordinary expiry, revocation and local admission still govern whether unstarted work may execute. $\square$

This argument covers interleaving extension with old and new dispatch because growth neither removes an allocation nor resets its execution identity. It requires a single protected head for the pool and a single native custody domain for each task. A copied database or a second independently provisioned graph does not inherit the exclusivity of the first. The result bounds declared commitments; it supplies neither financial backing nor a proof of task meaning. The native trajectory demonstrates conditional progress for a new worker and an original unstarted worker, without asserting termination of an arbitrary swarm. The S1 argument, like D1’s, is not a mechanized Rust refinement.

5.3 Funded obligations

For each funding source u , let D_u be its fixed deposited endowment, A_u its unallocated balance, L_u its locked allocations, P_u its paid outflow and R_u its refunded outflow. Returned cash remains recorded as an outflow in this accounting interval; a later deposit opens a new interval. The conservation invariant is

$$D_u = A_u + L_u + P_u + R_u, \quad A_u, L_u, P_u, R_u \geq 0. \quad (1)$$

Proposition 4 (Bounded funded exposure). *If reservation moves $q \leq A_u$ from available to locked funds, and payment or refund moves an allocation's locked amount once to its corresponding outflow, every reachable state satisfies (1). Total paid and refunded outflow is at most the original deposit.*

Proof. The initial state is $(D_u, 0, 0, 0)$. Reservation subtracts and adds the same quantity within the sum. Payment and refund do the same across a different pair of accounts. Their guards preserve nonnegativity. Other transitions do not change money. Induction over the transition sequence proves the equality and bound. This argument requires the implementation to identify allocations uniquely and couple each state transition with its actual transfer. $\square$

Proposition 5 (Earned child independence). *Suppose each work edge has a distinct allocation with its own immutable terms, an earned claim can only remain payable or become paid, and a transition on one edge cannot modify another's allocation. Then any sequence of parent failure, refund or completion transitions leaves an earned unpaid child collectible under its original terms.*

Proof. A transition concerning another edge leaves the child unchanged. A transition on the child preserves the set $\{\text{Payable}, \text{Paid}\}$. Induction therefore excludes child refund after acceptance. Distinct funding prevents a parent transfer from consuming the child's reserve. Eventual collection further requires availability, an exercisable beneficiary key and transaction inclusion. $\square$

A small Lean specification checks conservation over arbitrary finite account traces, the paid-plus-refunded bound, persistence of the earned states, and noninterference of distinct map entries. A finite Python explorer checks concrete timing and role guards; selected traces are replayed against Solidity. These are complementary checks. There is no machine-checked refinement from the Rust kernel or Solidity contract to the Lean specification, and the arithmetic proof does not authenticate a signer or prove that a token is honest.

The capital cost follows directly. If E_{open} contains the distinct unsettled obligations, their reserve is $\sum_{e \in E_{\text{open}}} q_e$. A chain of promised future receipts cannot reduce this amount under F1. The example locks 160 units while the two obligations coexist, although the buyer's task costs 100. Short milestones may reduce capital duration; they do not eliminate capital needs. Capital efficiency must be measured alongside success rate.

A payer-only signature cannot supply the missing exclusivity. Consider two receivers that see individually valid promises against the same deposit and have no common state or other non-equivocation mechanism. Each receiver's view is indistinguishable from a run in which it is the only payee. If both accept on that evidence alone, the two promises can exceed the deposit. The argument identifies a missing shared fact; a conventional trusted ledger, a qualified rollback-resistant authority or a settlement chain can supply it. It is not an impossibility result about OAuth, A2A, capabilities or ordinary escrow.

6 Evidence and comparison

We distinguish current D1 and S1 qualification from retained F1 experiments and September process evidence. Each has a source inventory; historical tests are checked against their preserved source tree rather than attributed to modified native code. All reported executions use author-controlled infrastructure. Fixture balances and development-chain tokens have no economic value.

6.1 Dynamic delegation in the native kernel

The Rust allocator implements root creation, signed subdivision, revisioned selection, sealing and portable verification. The native guard is an opt-in public installation API. Sixteen allocator tests cover recursive holders, concurrent sibling overcommit, selection/dispatch races, reopen, malformed predicates, expiry, integer bounds, authority substitution and offer reuse against changed terms, cross-owner namespace replay and persistent allocator identity. The full workflow crate run also exercises its existing tests; the artifact does not count those as new delegation experiments.

Eight substantive native tests cover dynamic provider choice, exact request and capability binding, offline-allocator execution, receiver substitution, output rejection before and after transforms, the configured receiver clock, required durable custody, and actual process-kill recovery with useful sibling progress. The native runner reports nine test labels because one is the subprocess worker. Recovery retains the original request and produces zero new tool dispatches after the selected cut. Rejected output yields a signed zero-charge denial in the qualified fixture.

The runnable example constructs a tree after startup, delegates through a second holder, replaces one selected provider before sealing, and executes through the replacement’s kernel. Reopening its independent native store returns the same receipt without another call. A sibling then completes, and fixture balances conserve their initial 1,000 units. Receivers hold separate keys and stores under one administrator; no remote network interoperability, independent operator or LLM planning result follows from this example.

A semantic negative test exposed an offer that authenticated only a slot name: the offer could be reused with different acceptance terms. Binding the full slot digest closes that case; the failing and passing outputs are retained. Fresh review also exposed signed mutations crossing honest allocator namespaces; binding the allocator, root and slot closes subdivision and selection replay. A further native regression found that a guard bypassed the configured clock; the guard now uses the kernel’s fenced authority clock. Compiler and fixture-configuration failures are retained separately and are not presented as protocol counterexamples. The qualification also reruns the existing three-owner native regression, checks Rust lints and formatting, runs the example and tests artifact provenance. These are bounded engineering checks, not a statistical estimate of production reliability.

6.2 Evolution through existing swarm and treaty authority

Five new verifier tests exercise checked growth, individually valid re-signed rewrites, caller-owned time and trust, duplicate attempts and budget bounds. Four store tests cover exact history after reopen, rollback after a rejected extension, simultaneous proposals through separate SQLite connections, and corrupted historical data. The race produces exactly one successor.

Three native trajectories exercise the effect boundary. First, an original worker executes, the graph grows, and the added worker executes. After reopen, replay returns the exact original receipt and physical claim; a fresh request for its consumed continuation is denied even though its ordinary

capability still has invocation headroom. Second, an original unstarted worker executes under its historical graph after extension. Third, a combined bilateral-treaty and swarm operation retains its three physical resources across extension and restart. The test explicitly revalidates the original claim, rejects a request with treaty context removed, and replays without another effect.

The qualification runs the complete swarm-authority and runtime-core crate suites, their all-target lints and formatting, alongside D1’s checks. These are tests of one implementation under author-controlled stores. They establish no new consensus mechanism, automatic receiver migration or independently operated network. The companion preserves the initial compiler and fixture failures separately from terminal passing evidence.

6.3 Retained funded evidence

Experiment	Result	Boundary
Finite claim graph	5,508 states	Two prefunded edges, fixed symbolic actors and nine timestamps
Transition checks	132,192	Conservation, terminal consistency, denial stability and uncertainty
Contract suite	143 passing	Compiled bytecode on a private development chain; includes 118 representative traces
Ordinary SQL comparator	312 matched steps	The same 118 traces; sequential trusted database
Upstream ERC-8183	76 passing	Unmodified upstream suite at the pinned commit
Paired escrow cases	20 passing	Same token, zero fees, capital and evaluator trust; local Cancun EVM
Lean specification	7 named results	Abstract transitions; no implementation refinement

Table 1: Retained F1 evidence at its recorded sources. The build checks the results and hashes; these counts are separate from current D1/S1 qualification.

6.4 Transitions and negative controls

The finite graph starts with a three-unit parent and a two-unit child. It explores alternative submissions, conflicting and unauthorized decisions, payment, refund, expiry and execution uncertainty over the selected timestamps. Every reachable transition is checked against the invariants. Exploration is exhaustive only within this bounded alphabet and starting configuration. Funding and failed-transfer boundaries have separate unit tests.

The 118 shortest representative financial traces cover selected state-transition classes. The Solidity runner compares contract states, commitments, decisions and actual token balances with the model after each step. It does not replay all 132,192 graph transitions against bytecode. Additional contract tests exercise failed and asymmetric transfers, unauthorized calls, duplicate funding, replay, verifier changes and delayed withdrawal. No confidence interval for deployed failure rates can be derived from this deterministic regression suite.

Two negative controls test whether the harness can detect the property it claims to enforce. A deliberately weakened model allows a payable job to expire; the explorer finds the sequence submission, acceptance, refund. A corresponding in-memory mutation of the compiled escrow is rejected by the earned-withdrawal regression. Neither negative control changes the source contract. Separately, the older escrow characterization exhibits a timely certificate losing payment eligibility

at expiry. That historical contract has different semantics; the paper does not describe it as an attempted equivalent implementation of the new rule.

6.5 An ordinary transactional alternative

We implement the monetary state machine using ordinary SQL transactions and conditional updates, without Chio kernel code. It receives the same role identities, deadlines and predicate outcomes as the model. It checks every observable snapshot and source balance along the same representative traces. The result is 312 matched steps and no mismatch. Additional tests preserve an uncollected child’s payment after parent refund, deny reuse of its funds, and keep unknown execution distinct from verifier rejection.

This is evidence that the tested monetary safety is available through standard mechanisms. The comparator trusts the database operator; replacing it with the same external escrow gives an ordinary application access to the same scarce backing. We have not benchmarked that application’s full integration effort or implemented every native admission feature in it. Accordingly, the comparison supports neither a Chio speedup nor an impossibility claim against alternatives. Both implementations were written within this project, so it is also not an independently authored replication.

A later conditional-backing experiment gives a stronger conventional allocator the same knowledge of earned, unknown, fenced-absent and exclusively selectable future claims. The proposed structural bound and an independent exhaustive oracle match in all 4,095 finite profiles. Selection before dispatch saves capacity relative to reserving every alternative, but the conventional construction obtains exactly the same saving. The companion retains this rejected exclusive-capital hypothesis. D1 consequently claims a reusable implementation boundary, not a uniquely available conservation rule.

6.6 Comparison with upstream ERC-8183

We also test the existing agent-commerce implementation at upstream commit `142e669c1f`. Its 76 unchanged tests pass. We add 20 comparison tests, using the same mock token, zero fees, a 100-unit buyer and a separately capitalized 60-unit intermediary. Each escrow uses its native authorization and deadline rules; distinct trusted evaluator addresses make decisions. The artifact retains compiled-bytecode traces, compiler and dependency pins, and reproduction after relocation.

Both implementations pay the child after parent refund, reject funding two 60-unit jobs from 100, and deny spending the unsettled parent reserve. The upstream implementation also supports a stronger milestone case: it pays an approved 60-unit claim from one 100-unit job, then refunds the remaining 40 when the job is rejected. Chio’s fixed-price allocation has no partial-settlement entrapment. These results reject an exclusive claim to either funded agent jobs or surviving child payments.

The observed differences concern state-machine policy. Chio separates recorded acceptance from withdrawal, rejects new acceptance after its resolution cutoff, and preserves earned withdrawal through a funding pause or allowlist change. Upstream completion pays immediately; after its expiry grace period, completion can still win if refund has not executed. Upstream refunds are blocked while paused. These are properties of the pinned implementations, not expressiveness limits of ERC-8183, whose evaluators and hooks admit extensions. No gas, performance or security superiority follows from the comparison.

6.7 Execution, verification and security integration

The existing funded prototype binds actual Rust-produced output to a separately implemented Python check, signed custody, a claim and payment. Its workload inventories supported OpenAPI authentication declarations. It checks the listed properties, not the security of a running API. September evidence records payment, rejection, missing custody, an earned child after parent refund, transaction recovery and native execution witnesses. Its later process experiments isolate role keys and pass public evidence between separate roles.

We reconciled funded source 7755d3762b with security source 491f585e90 in integration commit 71e5cbc3bf. Their changes overlap in durable admission, recovery and evidence handling; compatibility cannot be inferred from their independent tests. The integrated evidence snapshot passes 95 native tests, including all six opt-in chain tests, and 20 selected process scenarios covering authenticated transport, bilateral isolation, earned-child recovery, refund resolution and custody. The final source differs only by removal of a redundant dereference, checked by strict Clippy, four affected unit tests and a fresh authenticated payout/refund run. All roles remain under author control. Broader formal-source mapping, file-hygiene, hosted and deployment qualification remain open. The companion retains the exact integration record and each remaining gate.

The deterministic workload deliberately makes acceptance easy to inspect. It does not demonstrate an asymmetry between expensive production and cheap verification: a buyer can often compute this inventory itself. There is no held-out repair corpus, economic-margin result, live-agent advantage or independently operated company trial in the reported evaluation. Those omissions bound the broader economy claim rather than the tested escrow behavior.

7 Relation to existing mechanisms

Miller’s robust composition addresses cooperation among potentially malicious distributed components using object capabilities [1]. Secure Linking uses certified properties and proof-carrying authentication to govern component linking [2]. Multiparty session types derive local checks from global interaction types and establish communication properties within their calculus [6]. These works precede the general ambition of composing independently controlled components. S1 contributes a concrete extension rule for Chio’s existing signed authority and native continuation custody. It does not introduce secure composition in general or establish the session-type progress guarantees for arbitrary agent programs.

Contract Net already organizes distributed work through announcements, bids and awards [7]. Agent Contracts unifies task specifications, resource constraints and success conditions, including delegated budget conservation [8]. Chio does not introduce contracting or hierarchical budget discipline. D1 connects such bounds to a sealed, portable selection and an exact native invocation; F1 connects accepted work to an obligation backed by a declared settlement authority.

Proof-carrying code makes supplied evidence checkable against a consumer’s safety policy [9]. Interface Automata models component assumptions, guarantees, compatibility and refinement [10]. These are important antecedents to locally checked composition. D1’s permit carries authenticated allocation evidence, not a proof of arbitrary program behavior. Its subset rules do not establish semantic refinement between parent and child tasks. A successful predicate check certifies only that predicate.

AgentBound places deterministic governance before execution and binds decisions to verifiable receipts [11]. That overlaps substantially with local authorization and audit. A receipt about

permitted behavior does not by itself reserve funds at a separate authority or decide application-specific output acceptance. Chio’s work contract explicitly joins those duties, while retaining their distinct assumptions.

A2A supplies tasks, messages, artifacts and discovery [12]; AP2 specifies signed commercial and payment mandates and their verification [13]. AP2 v0.2 leaves shopping-agent mandate delegation outside its scope and permits extensible authorization models. That is a scope choice, not evidence that its carriers cannot express this work contract. Likewise, OAuth token exchange leaves significant trust and token semantics to deployment profiles [14]. A receiver-issued credential and ordinary signed application fields can enforce receiver-owned authorization. Interoperability requires a defined profile and conformance tests, not a claim that other formats lack expressive power.

ERC-8183 is a particularly close construction: it specifies escrowed agent jobs, providers, evaluator-approved completion and expiry refunds [15]. The economic pattern is already present. The pinned upstream implementation additionally supports partial claims and relayable EIP-712 authorization; the specification and implementation differ in details. Section 6 tests their overlap with Chio directly. No new economic primitive follows from separating acceptance and withdrawal or from independently funding child jobs. ERC-8004 provides discovery, reputation and validation interfaces while leaving payments to other protocols [16]. A registry entry is evidence a receiver may consider, not a capability to use its resources. Both standards are drafts at the pinned snapshots.

Sagas explain how sequences of transactions use application-defined compensation [17]. A refund is such a financial action; it need not erase an already delivered effect. Our explicit unknown state prevents recovery from inventing a successful execution or a no-effect history. Durable workflows and transactional outboxes remain appropriate competing implementation techniques.

Contingent service payments and fair exchange have established constructions for linking payment to verification [18, 19]. F1 is weaker in an important respect: it relies on a selected verifier, rather than supplying a cryptographic proof of arbitrary useful work or complete fair exchange. Replacing that verifier with a proof system would require a restricted task language, cost measurements and a new analysis of availability and disclosure. The current implementation does not establish that extension.

The relevant comparison is thus a composition of competent implementations of these mechanisms. A missing field in a base specification is not an obstacle to that composition. Chio offers a particular contract and enforcement path; whether that common contract materially lowers the cost of changing partners remains the experiment that would distinguish protocol value from a large bespoke integration.

8 Where the argument stops

D1 has a trusted owner allocator and one protected custody domain per receiver key. It cannot prevent those trusted components from equivocating after state fork or rollback. Its offline permits require prior qualification and do not provide instantaneous global revocation. Sealing may strand unused capacity; expired, withheld or unresolved work has no automatic reclaim path. Output validation and recovery recheck contract expiry, so completion after that deadline may remain stranded. The implemented predicates and finite trees intentionally restrict expressiveness. Declared reader/effect attenuation still depends on complete host mediation. The profile does not track arbitrary information flow through tools, logs, errors and payment channels.

S1 conservatively retains every allocation and prior permission. Its storage and verification costs

grow with the retained graph and history; no scalability or hostile-load result is reported. Policy changes, revocation-epoch replacement, task retirement, budget reclaim and receiver migration require explicit lifecycle rules beyond additive extension. The D1 and S1 trajectories do not yet bind their distinct allocation representations to F1’s escrow in a single evaluated execution. Doing so requires one explicit owner for each scarce resource and preservation of its identity at every handoff.

The two enduring dependencies in F1 are the acceptance authority and the settlement domain. Parties may choose them without choosing a shared task scheduler, but cannot dispense with their assumptions. An escrow administrator can restrict new participation; an asset issuer may freeze transfers; a verifier may censor or falsely approve. Immutable earned-state rules alone do not defeat those failures. No new token, global reputation system or universal identity registry is necessary for the demonstrated profile.

Verifiable work means work with an agreed check, not universally decidable quality. A test suite may miss a vulnerability. A proof may prove the wrong specification. A correct declared-authentication inventory may have little commercial value. The buyer must choose an acceptance procedure fit for its purpose and a disclosure policy it can tolerate. Once disclosed, data cannot be recalled by revoking a capability.

Evidence integrity is narrower than availability. A content hash detects substitution but cannot recover withheld bytes. A custody signature records a promise to retain data; it does not prove future service or erase storage costs. Likewise, a historical signed receipt can remain valid after authority expires, while fresh dispatch must still consult current authority and revocation state.

Our safety argument abstracts serial atomic rail transitions, and the local experiments use a private development chain or an owned chain fixture. They do not establish public-chain finality, independent observers, geographic failure tolerance, sustained denial-of-service resilience or hostile administration of the honest party’s kernel. The Lean model and finite exploration have explicit boundaries; passing them does not prove the implementation as a whole.

The next decisive evaluation is an independently implemented provider and separately administered participants completing useful held-out work from the published contract. It must allow the alternative the same escrow, checker, models, budgets and ordinary application extensions. It should measure integration changes, failed-attempt cost, capital duration, verifier cost, recovery effort and buyer usefulness. Without those results, neither lower integration cost nor a profitable autonomous supply chain is an empirical conclusion of this paper.

9 Conclusion

An agentic kernel must let a program change its collaborators without letting its planner rewrite existing obligations. Chio makes that boundary concrete: delegate bounded capacity, seal a selected invocation, and extend a signed swarm while retaining prior commitments. The receiver applies its own authority; native custody preserves execution identity across graph versions and failure. Under the stated protected-state assumptions, all declared allocations in an installed lineage remain bounded by its original pool. A separate funded profile preserves recorded earned payments through parent failure.

The Rust implementation joins this rule to existing swarm authority, bilateral treaties and recovery. Its native trajectories establish growth with preserved old work and replay custody. Independent useful-work and integration trials remain necessary to measure the value of adopting the common interface. The result gives an agentic web a concrete execution contract to implement and evaluate:

participants may extend the program while each owner retains local authority and each issued commitment retains its history.

References

- [1] Mark Samuel Miller. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. PhD thesis, Johns Hopkins University, 2006. URL https://worrydream.com/refs/Miller_2006_-_Robust_Composition.pdf.
- [2] Eunyoung Lee and Andrew W. Appel. Secure Linking: a Framework for Trusted Software Components (Extended Version). Technical Report TR-663-02, Princeton University, 2002. URL <https://www.cs.princeton.edu/techreports/2002/663.pdf>.
- [3] Satoshi Nakamoto. Bitcoin: A Peer-to-Peer Electronic Cash System, 2008. URL <https://bitcoin.org/bitcoin.pdf>.
- [4] Anders Rundgren, Bret Jordan, and Samuel Erdtman. JSON Canonicalization Scheme (JCS). RFC 8785, 2020. URL <https://www.rfc-editor.org/rfc/rfc8785>.
- [5] Remco Bloemen, Leonid Logvinov, and Jacob Evans. EIP-712: Typed Structured Data Hashing and Signing, 2017. URL <https://eips.ethereum.org/EIPS/eip-712>.
- [6] Kohei Honda, Nobuko Yoshida, and Marco Carbone. Multiparty Asynchronous Session Types. In *Proceedings of POPL*, pages 273–284, 2008. doi: 10.1145/1328897.1328472. URL <https://mrg.cs.ox.ac.uk/publications/multiparty-asynchronous-session-types/>.
- [7] Reid G. Smith. The Contract Net Protocol: High-Level Communication and Control in a Distributed Problem Solver. *IEEE Transactions on Computers*, C-29(12):1104–1113, 1980. doi: 10.1109/TC.1980.1675516.
- [8] Qing Ye and Jing Tan. Agent Contracts: A Formal Framework for Resource-Bounded Autonomous AI Systems. arXiv:2601.08815v3, 2026. URL <https://arxiv.org/html/2601.08815v3>.
- [9] George C. Necula. Proof-Carrying Code. In *Proceedings of POPL*, pages 106–119, 1997. URL <https://homes.cs.washington.edu/~mernst/teaching/6.893/readings/necula-popl97.pdf>.
- [10] Luca de Alfaro and Thomas A. Henzinger. Interface Automata. In *Proceedings of ESEC/FSE*, pages 109–120, 2001. URL https://web.cs.wpi.edu/~heineman/html/teaching_CS562/p109-de_alfaro.pdf.
- [11] Anuj Kaul, Qianlong Lan, and Pranay Gupta. Behavioral Governance for Autonomous AI Agents: The AgentBound Framework. arXiv:2606.30970v2, 2026. URL <https://arxiv.org/html/2606.30970v2>.
- [12] A2A Project. Agent2Agent Protocol Specification, 2026. URL <https://github.com/a2aproject/A2A/blob/65dadbd9b900fe2d970e05416eb432a91347cd2a/docs/specification.md>. Source pin 65dadbd9b900fe2d970e05416eb432a91347cd2a.
- [13] AP2 Contributors. Agentic Payment Protocol, version 0.2, 2026. URL <https://ap2-protocol.org/ap2/specification/>. Source pin and retrieval hash in the artifact’s sources.json.

- [14] Michael B. Jones, Anthony Nadalin, Brian Campbell, John Bradley, and Chuck Mortimore. OAuth 2.0 Token Exchange. RFC 8693, 2020. URL <https://www.rfc-editor.org/rfc/rfc8693>.
- [15] Davide Crapis, Bryan Lim, Tay Weixiong, and Chooi Zuhwa. ERC-8183: Agentic Commerce, 2026. URL <https://eips.ethereum.org/EIPS/eip-8183>. Draft, source snapshot 365b4c02879f3e882b91281d42b4f57b406205e9.
- [16] Marco De Rossi, Davide Crapis, Jordan Ellis, and Erik Reppel. ERC-8004: Trustless Agents, 2025. URL <https://eips.ethereum.org/EIPS/eip-8004>. Draft, source snapshot 365b4c02879f3e882b91281d42b4f57b406205e9.
- [17] Hector Garcia-Molina and Kenneth Salem. Sagas. Technical Report CS-TR-070-87, Princeton University, 1987. URL <https://www.cs.princeton.edu/techreports/1987/070.pdf>.
- [18] Matteo Campanelli, Rosario Gennaro, Steven Goldfeder, and Luca Nizzardo. Zero-Knowledge Contingent Payments Revisited: Attacks and Payments for Services. Cryptology ePrint Archive, 2017/566, 2017. URL <https://eprint.iacr.org/2017/566>.
- [19] Stefan Dziembowski, Lisa Ekey, and Sebastian Faust. FairSwap: How to Fairly Exchange Digital Goods. Cryptology ePrint Archive, 2018/740, 2018. URL <https://eprint.iacr.org/2018/740>.